COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithm	**SECTION:**	CPE21S3
DATE PERFORMED:	10/20/26	**DATE SUBMITTED:**	10/20/26
NAME:	Dimaguiba, Lorenzo James M. Urbano, Vince Andrew F.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HANDS-ON ACTIVITY NO. 14.1 Algorithm Complexity

1. PROCEDURE OUTPUT

	Best Case	Worst Case	Analysis
Insertion Sort	O(n)	$O(n)^2$	The best case of insertion sort happens when the given array is already sorted. The worst case on the other hand has the time complexity of $O(n)^2$ which happens if the given array is given in reverse meaning the sorting algorithm has to check each value inside the array one by one.
Merge Sort	O(nlogn)	O(nlogn)	One of the more stable sorting algorithms of the bunch. With a divide-and-conquer approach, it continually halves the given list or array into smaller subvectors, eventually, sorting the list back into one whole in the end. Since the sorting part of the algorithm is independent of the data's order, its best and worst case scenarios are basically the same (O(nlogn)).

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

	Insertion Sort	Merge Sort

Best Case	100:
	`437400 nanoseconds`
	
	1000:
	`499500 nanoseconds`
	
	10,000:
	`493000 nanoseconds`
		`162400 nanoseconds`
		
		1000:
		`119500 nanoseconds`
		
		10,000:
		`415600 nanoseconds`

Worst Case	100: 619300 nanoseconds 1000: 643100 nanoseconds 10,000: 1768400 nanoseconds	520000 nanoseconds 1000: 453900 nanoseconds 10,000: 1404200 nanoseconds

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

Analysis:
The code makes use of multithreading to **ONLY DEMONSTRATE** parallelism. One half is sorted by the insertion sort, the other with the merge sort. This leaves us with two sorted halves. Once both halves are sorted a final insertion sort two halves together, which will be faster as less shifts have to be done. Based on the chrono results, the merge sort algorithm tends to perform better than the insertion sort in terms of complexity, with logarithmics having lesser numbers of computations.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Input Size (N)	Elapsed Time for Binary Search (Worst-Case)	Elapsed Time for Linear Search (Worst-Case)
1,000	Linear Search: Searching is successful Linear Search Function executed for 319449 nanoseconds	Binary Search: Searching is successful Binary Search Function executed for 209020 nanoseconds
10,000	Linear Search: Searching is successful Linear Search Function executed for 231130 nanoseconds	Binary Search: Searching is successful Binary Search Function executed for 62180 nanoseconds
100,000	Linear Search: Searching is successful Linear Search Function executed for 409420 nanoseconds	Binary Search: Searching is successful Binary Search Function executed for 129860 nanoseconds

Code Analysis:
During the supplementary, we have used binary and linear search algorithms. The linear search function works by searching the arrays one by one for the value wanted by the user inside the array that was made. Binary search on

the other hand, works by dividing the array in half which makes it much more efficient for finding the array especially on large amounts of data sets. We have also imported the thread library which can be used for threading and .join() for multithreading. The function of .join() is to block the thread from working which means it can block the other code until the line finishes. Larger datasets require more time to search for the value needed by the user. We have also implemented a bubble sort since the binary search requires the array to be sorted first before it can search.

Conclusion:

Dimaguiba:
The activity pushed us to apply our newfound knowledge with both multithreading, in order to run several functions at the same time, and in order to compare performances depending on time complexity and the time elapsed to finish a task. We are able to apply it to both our procedures and supplementary activities. This can be useful and aids in our decision-making to decide which algorithms and functions are most appropriate for whatever future project we are to perform.

Urbano:
During this activity, I have learned how to use chrono more and how other algorithms have their own time and space complexity. Other algorithms also differ from how much time they need to execute. I have also learned more about multithreading functions, especially on how to use them effectively. I also understand why we need to pick the best sorting and searching algorithm based on the size of the array as it has a huge effect on how much time it takes to search and sort an array.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **N/A**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.


T.I.P
TECHNOLOGICAL INSTITUTE OF THE PHILIPPINES
COMPUTER ENGINEERING
QUEZON CITY